Class Notes: Advanced DAG Features & Task Management

---

1. How to use SubDAGs? (And Why You Probably Shouldn't)

SubDAGs were the original Airflow construct for grouping multiple tasks into a single, reusable unit. A SubDAG is a DAG that is embedded inside another "parent" DAG.

- **How it worked:** You would use the SubDagOperator, passing it the subdag function which defined the inner DAG.
- **The Fatal Flaw:** SubDAGs have a critical design issue: **they execute using their own executor, and by default, this was the** SequentialExecutor. This meant that all tasks inside the SubDAG would run one after another, even if the parent DAG was using the LocalExecutor or CeleryExecutor, completely killing parallelism and becoming a major performance bottleneck.
- **Legacy Status:** SubDAGs are now **deprecated** and should be considered a legacy feature. They are covered here for historical context and to understand why the superior alternative, **TaskGroups**, was created.

Key Takeaway: Avoid SubDAGs. Use TaskGroups instead.

---

2. Group tasks with SubDAGs (Historical Example)

For educational purposes, here is what the syntax looked like:

File: dags/parent_dag.py

```
from airflow import DAG
from airflow.operators.subdag import SubDagOperator
from datetime import datetime


def subdag_function(parent_dag_name, child_dag_name, args):
    """
    Function to generate a SubDAG.
    This was the required pattern.
    """
    with DAG(
        dag_id=f"{parent_dag_name}.{child_dag_name}", # SubDAGs had compound IDs
        default_args=args,
        schedule_interval=None, # SubDAGs are triggered by their parent
    ) as subdag:
```

```
    # ... define tasks inside the SubDAG (e.g., Task_A, Task_B, Task_C) ...

    Task_A >> Task_B >> Task_C

  return subdag

with DAG(
  dag_id="parent_dag",
  start_date=datetime(2023, 1, 1),
  schedule_interval="@daily"
) as dag:

  start = DummyOperator(task_id="start")

  # Using the SubDagOperator
  process_data = SubDagOperator(
    task_id="process_data",
    subdag=subdag_function("parent_dag", "process_data", default_args),
    # This would silently force tasks to run sequentially!
  )

  end = DummyOperator(task_id="end")

  start >> process_data >> end
```

---

3. TaskGroups (The Modern Replacement)

TaskGroups are the modern, recommended way to group tasks in Airflow. They solve all the problems of SubDAGs:

- **No Performance Hit:** Tasks inside a TaskGroup are just regular tasks in the parent DAG. They use the same executor and can run in parallel with each other and with tasks outside the group.
- **Simpler Syntax:** They are created using a context manager (with TaskGroup(...):), making the code much cleaner.

- **UI Benefits:** In the Airflow UI (Graph View), the group can be **collapsed** into a single node to reduce visual clutter or **expanded** to see the detailed tasks inside.

Key Difference: A TaskGroup is a *visual and organizational* grouping. A SubDAG was a *functional and execution* grouping.

---

4. Group tasks with TaskGroups

Here's how to create the same functionality as the SubDAG example, but correctly using a TaskGroup.

File: dags/parent_dag_with_group.py

```
from airflow import DAG
from airflow.operators.dummy import DummyOperator
from airflow.operators.python import PythonOperator
from airflow.utils.task_group import TaskGroup # Import the TaskGroup class
from datetime import datetime

def print_task_type(**kwargs):
    """A simple function to print the task type and ID."""
    print(f"This is task: {kwargs['task_id']}")

with DAG(
    dag_id="advanced_dag_with_groups",
    start_date=datetime(2023, 1, 1),
    schedule_interval=None,
    catchup=False
) as dag:

    start = DummyOperator(task_id="start")

    # Create a TaskGroup for the ETL process
    with TaskGroup("etl_pipeline", tooltip="Extract, Transform, Load data") as etl_pipeline:

        # Tasks inside the group are indented
        extract = PythonOperator(
```

```
        task_id="extract",
        python_callable=print_task_type,
        op_kwargs={'task_id': 'extract'}
    )

    transform = PythonOperator(
        task_id="transform",
        python_callable=print_task_type,
        op_kwargs={'task_id': 'transform'}
    )

    load = PythonOperator(
        task_id="load",
        python_callable=print_task_type,
        op_kwargs={'task_id': 'load'}
    )

    # Define dependencies WITHIN the group
    extract >> transform >> load

# Create another TaskGroup
with TaskGroup("reporting", tooltip="Generate reports") as reporting:
    generate_report = DummyOperator(task_id="generate_report")
    send_email = DummyOperator(task_id="send_email")

    generate_report >> send_email

end = DummyOperator(task_id="end")

# Define dependencies BETWEEN groups and other tasks
start >> etl_pipeline >> reporting >> end
```

5. Sharing data between tasks with XComs

We covered XCom basics earlier. Let's dive deeper into the mechanics.

- **Automatic XCom:** The return value of any Python function called by a PythonOperator, @task decorated function, or other operators is **automatically pushed** to XCom. The key is return_value.
- **Manual XCom:** You can manually push and pull using the ti (Task Instance) object available in the context.
- **Custom Keys:** Always use a descriptive key when pushing manually to avoid conflicts.

Example: Explicit XCom Push and Pull

```
def pushing_function(**kwargs):
    ti = kwargs['ti']
    # Push a value with a custom key
    ti.xcom_push(key='filename', value='my_processed_data.csv')
    return 42  # This is automatically pushed to key 'return_value'


def pulling_function(**kwargs):
    ti = kwargs['ti']
    # Pull the manually pushed value
    filename = ti.xcom_pull(task_ids='push_task', key='filename')
    # Pull the automatically pushed return value
    count = ti.xcom_pull(task_ids='push_task')
    print(f"Using file {filename}, count was {count}")
```

Important: XCom is for small data (metadata, file paths, small results). For large data, pass a path and have the next task read from the source (S3, database, etc.).

---

6. Choosing a specific path in your DAG (Branching)

A common pattern is to choose which task(s) to run next based on the result of an upstream task. This is done using **branching operators**. The most common is the BranchPythonOperator.

- **How it works:** The branch operator's callable function does **not** perform the work; it only **returns the** task_id (or a list of task_ids) of the *next* task(s) that should be executed. All other paths are skipped.

Example: Process data differently based on its source.

```
from airflow.operators.python import BranchPythonOperator
```

```
def _choose_branch(**kwargs):
    ti = kwargs['ti']
    # Pull data from a previous task to make a decision
    data_source = ti.xcom_pull(task_ids='get_data_source')
    if data_source == 'api':
        return 'process_api_data' # task_id to run next
    elif data_source == 'database':
        return 'process_db_data'
    else:
        return 'handle_unknown_source'

choose_branch = BranchPythonOperator(
    task_id='choose_branch',
    python_callable=_choose_branch
)

get_data_source = DummyOperator(task_id='get_data_source')
process_api_data = DummyOperator(task_id='process_api_data')
process_db_data = DummyOperator(task_id='process_db_data')
handle_unknown_source = DummyOperator(task_id='handle_unknown_source')

get_data_source >> choose_branch
# The branch operator will trigger ONLY ONE of the following paths
choose_branch >> process_api_data
choose_branch >> process_db_data
choose_branch >> handle_unknown_source
```

---

7. Executing a task according to a condition

Sometimes you don't need a full branch but just want to conditionally execute a single task. This can be achieved within a PythonOperator or by using the ShortCircuitOperator.

- ShortCircuitOperator**:** A special operator. If its callable function returns False, it skips all downstream tasks. If it returns True, it continues as normal.

Example: Only run weekend reporting on Saturdays.

```
from airflow.operators.python import ShortCircuitOperator


def _is_weekend(**kwargs):
    execution_date = kwargs['execution_date']
    # Return True (continue) only if it's Saturday (5) or Sunday (6)
    return execution_date.weekday() >= 5


weekend_check = ShortCircuitOperator(
    task_id='weekend_check',
    python_callable=_is_weekend
)


generate_weekend_report = DummyOperator(task_id='generate_weekend_report')


weekend_check >> generate_weekend_report
# If it's a weekday, generate_weekend_report and all tasks after it will be skipped.
```

---

8. Trigger rules or how tasks get triggered

By default, a task is triggered only when **all of its upstream tasks have succeeded**. This is the all_success trigger rule. However, this isn't always the desired behavior, especially with branching.

The trigger_rule parameter on a task allows you to change this logic.

- **Common Trigger Rules:**
    - all_success: (Default) All upstream tasks have succeeded.
    - all_failed: All upstream tasks are in a failed or upstream_failed state.
    - one_failed: At least one upstream task has failed. It triggers immediately without waiting for all upstream tasks to finish.
    - one_success: At least one upstream task has succeeded.
    - none_failed: All upstream tasks have not failed (i.e., they succeeded or were skipped).
    - none_skipped: No upstream task is in a skipped state.
    - dummy: No dependency logic at all; runs immediately. Useful for grouping tasks.

---

9. Fixing the BranchPythonOperator with trigger rules

A classic problem occurs with the branch operator. Look at this flawed setup:

python

```
get_data >> choose_branch >> [process_a, process_b] >> join_task
```

- If choose_branch returns process_a, then process_b is skipped.
- The join_task has two upstream tasks: process_a and process_b.
- By default, join_task uses trigger_rule='all_success'.
- But one of its upstream tasks (process_b) was skipped! So join_task will never run.

The Fix: Change the trigger_rule for join_task to none_failed or none_skipped. This means "run as long as no upstream task failed, even if some were skipped."

```
join_task = DummyOperator(
    task_id='join_task',
    trigger_rule='none_failed' # The crucial fix
)
```

This tells join_task to run as long as process_a succeeded and process_b was skipped (which is not a failure state).